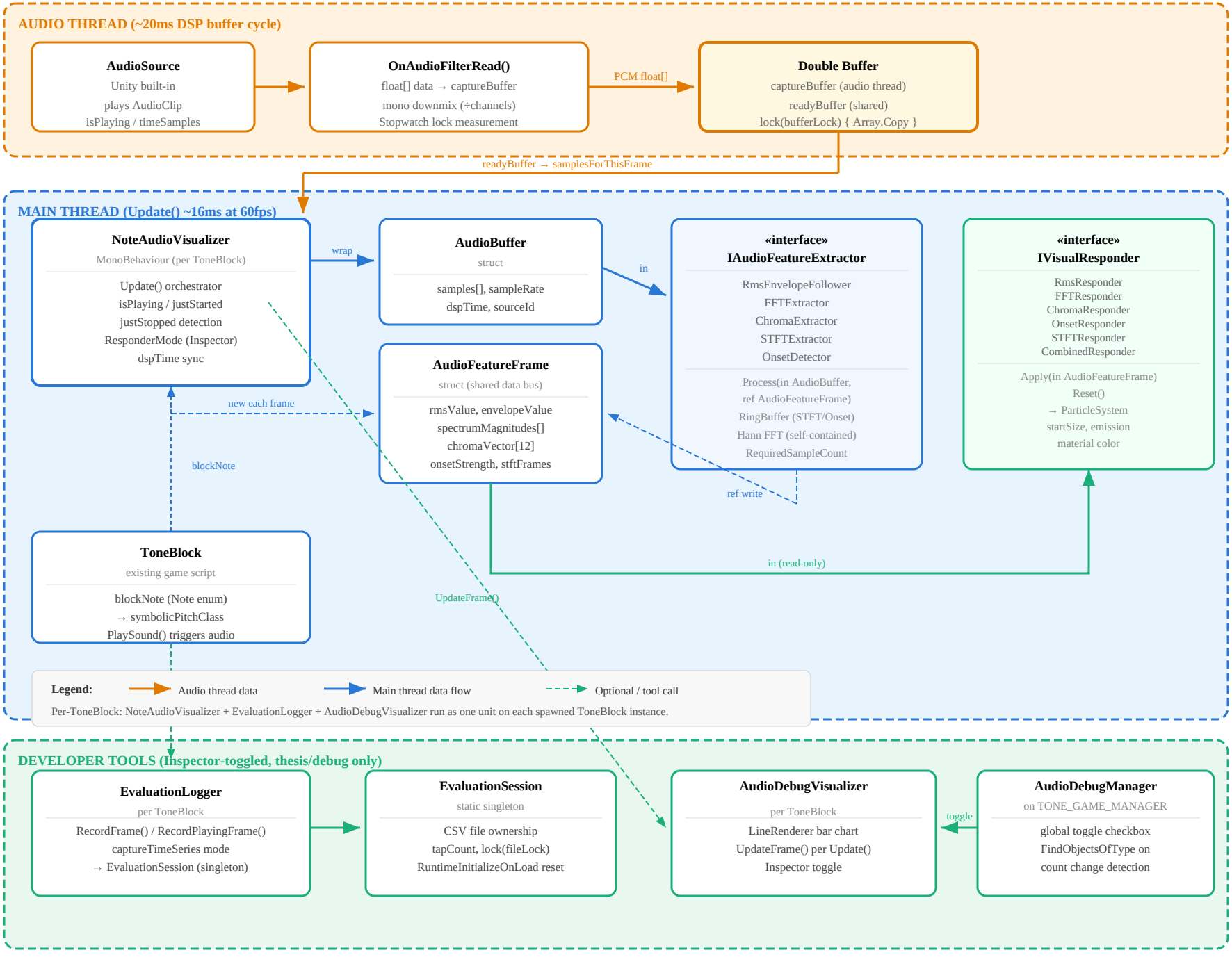
AUDIO THREAD (~20ms DSP buffer cycle)
AudioSource
Unity built-in
plays AudioClip
isPlaying / timeSamples
OnAudioFilterRead()
float[] data → captureBuffer
mono downmix (÷channels)
Stopwatch lock measurement
PCM float[]
Double Buffer
captureBuffer (audio thread)
readyBuffer (shared)
lock(bufferLock) { Array.Copy }
readyBuffer → samplesForThisFrame
MAIN THREAD (Update() ~16ms at 60fps)
NoteAudioVisualizer
MonoBehaviour (per ToneBlock)
Update() orchestrator
isPlaying / justStarted
justStopped detection
ResponderMode (Inspector)
dspTime sync
wrap
AudioBuffer
struct
samples[], sampleRate
dspTime, sourceId
in
«interface»
IAudioFeatureExtractor
RmsEnvelopeFollower
FFTExtractor
ChromaExtractor
STFTExtractor
OnsetDetector
Process(in AudioBuffer,
ref AudioFeatureFrame)
RingBuffer (STFT/Onset)
Hann FFT (self-contained)
RequiredSampleCount
«interface»
IVisualResponder
RmsResponder
FFTResponder
ChromaResponder
OnsetResponder
STFTResponder
CombinedResponder
Apply(in AudioFeatureFrame)
Reset()
→ ParticleSystem
startSize, emission
material color
new each frame
AudioFeatureFrame
struct (shared data bus)
rmsValue, envelopeValue
spectrumMagnitudes[]
chromaVector[12]
onsetStrength, stftFrames
blockNote
ref write
in (read-only)
ToneBlock
existing game script
blockNote (Note enum)
→ symbolicPitchClass
PlaySound() triggers audio
UpdateFrame()
Legend:
Audio thread data
Main thread data flow
Optional / tool call
Per-ToneBlock: NoteAudioVisualizer + EvaluationLogger + AudioDebugVisualizer run as one unit on each spawned ToneBlock instance.
DEVELOPER TOOLS (Inspector-toggled, thesis/debug only)
EvaluationLogger
per ToneBlock
RecordFrame() / RecordPlayingFrame()
captureTimeSeries mode
→ EvaluationSession (singleton)
EvaluationSession
static singleton
CSV file ownership
tapCount, lock(fileLock)
RuntimeInitializeOnLoad reset
AudioDebugVisualizer
per ToneBlock
LineRenderer bar chart
UpdateFrame() per Update()
Inspector toggle
toggle
AudioDebugManager
on TONE_GAME_MANAGER
global toggle checkbox
FindObjectsOfType on
count change detection